

量子计算化学框架汇报

一、为什么计算化学需要量子计算

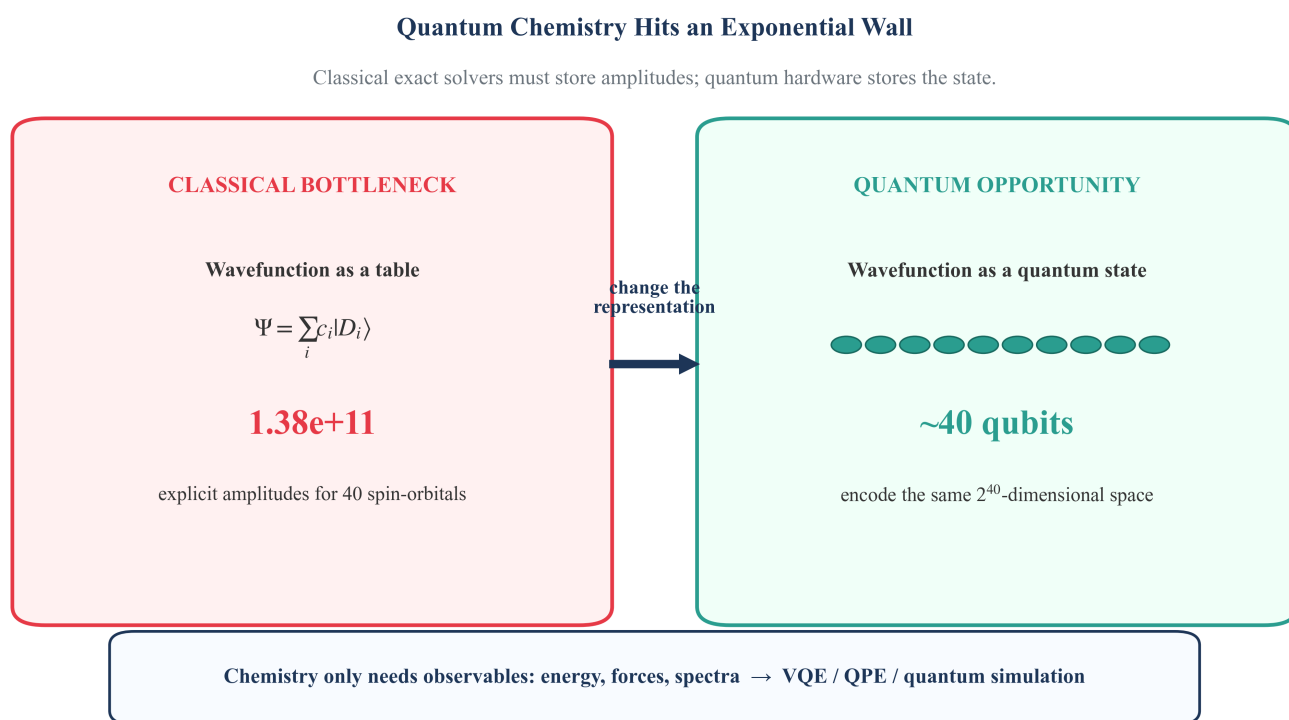
1.1 经典计算的"维数灾难"

FCI 要显式展开多电子波函数，活性轨道一多，Slater 行列式数就指数爆炸：

$$\dim \mathcal{H}_{\text{FCI}} = \binom{n_{\text{so}}}{N_e} \quad (1)$$

量子计算的机会：用约 n_{so} 个量子比特承载 $2^{n_{\text{so}}}$ 维态空间，只测能量等关键观测量。

尺度感：2 个 H_2O 就有 20 个电子；真实化学体系很快超出经典精确求解范围。



1.2 为什么量子计算机天然适合

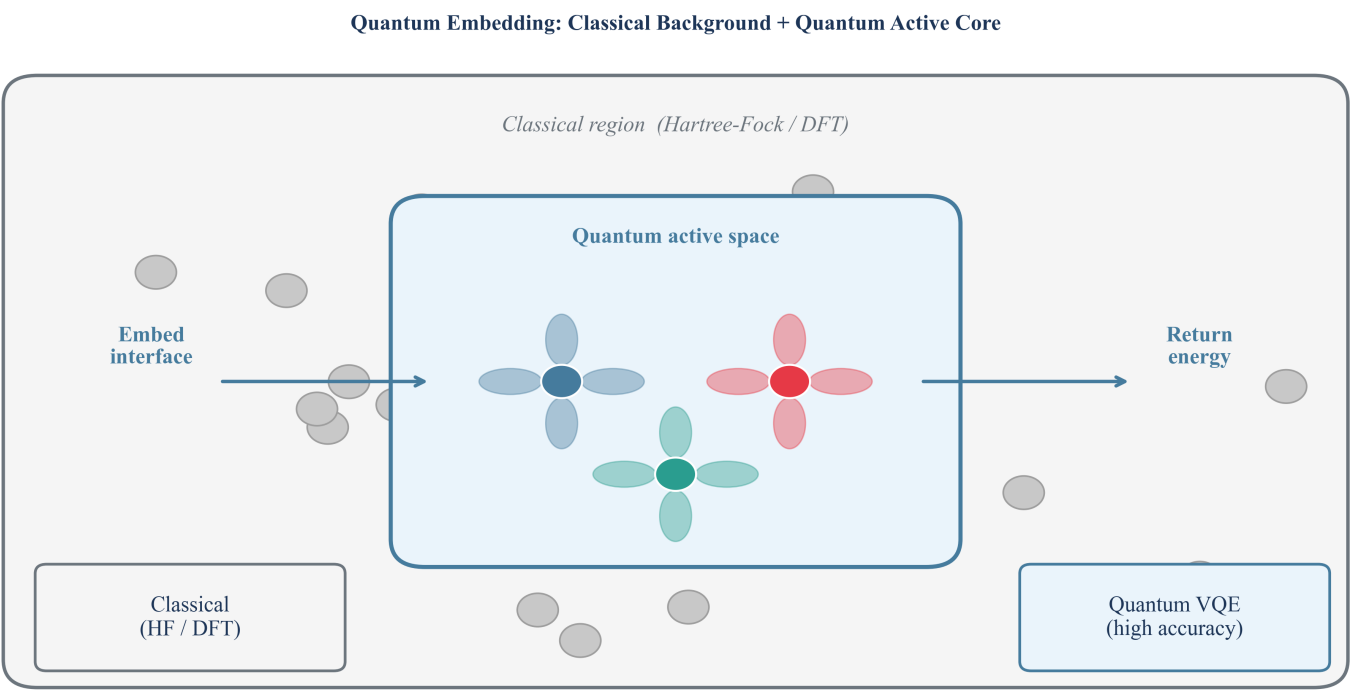
分子本身就是量子系统。量子比特（qubit）和电子的量子态"同构"——用量子计算机模拟量子系统。

1.3 工程目标

搭建严谨、可审计的编排层，让化学家能可靠地把分子问题翻译成量子电路，并把结果可信地用回化学。

二、分子的"量子内核"：活性空间与嵌入（Embedding）

现阶段量子计算机只有有限的量子比特（NISQ 时代）。如何用有限算力算大分子？



2.1 策略：经典与量子各司其职

区域	负责方	方法
外层骨架（灰色）	经典计算机	Hartree-Fock / DFT（低成本近似）
核心活性区（彩色轨道）	量子计算机	VQE / ADAPT-VQE（高精度求解）

三、竞品深度拆解：InQuanto vs Tangelo

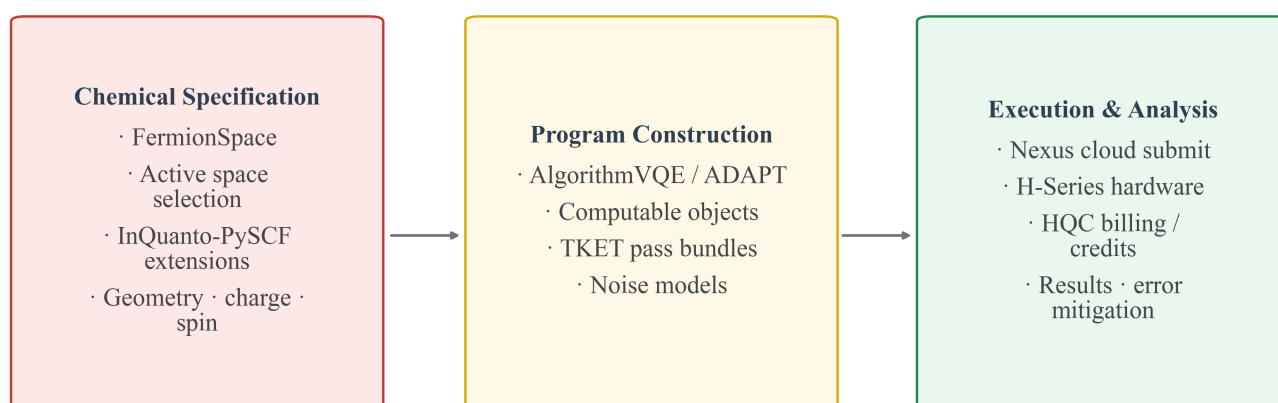
了解竞品是定位自身优势的前提。主要有两个标杆性竞品：

3.1 InQuanto：商业完整平台（Quantinuum）

InQuanto 是 Quantinuum 的商业量子化学平台，与 H-Series 离子阱、TKET 编译器和 Nexus 云深度集成。

3.1.1 三支柱 workflow

InQuanto — Three-Pillar Commercial Workflow (Quantinuum)



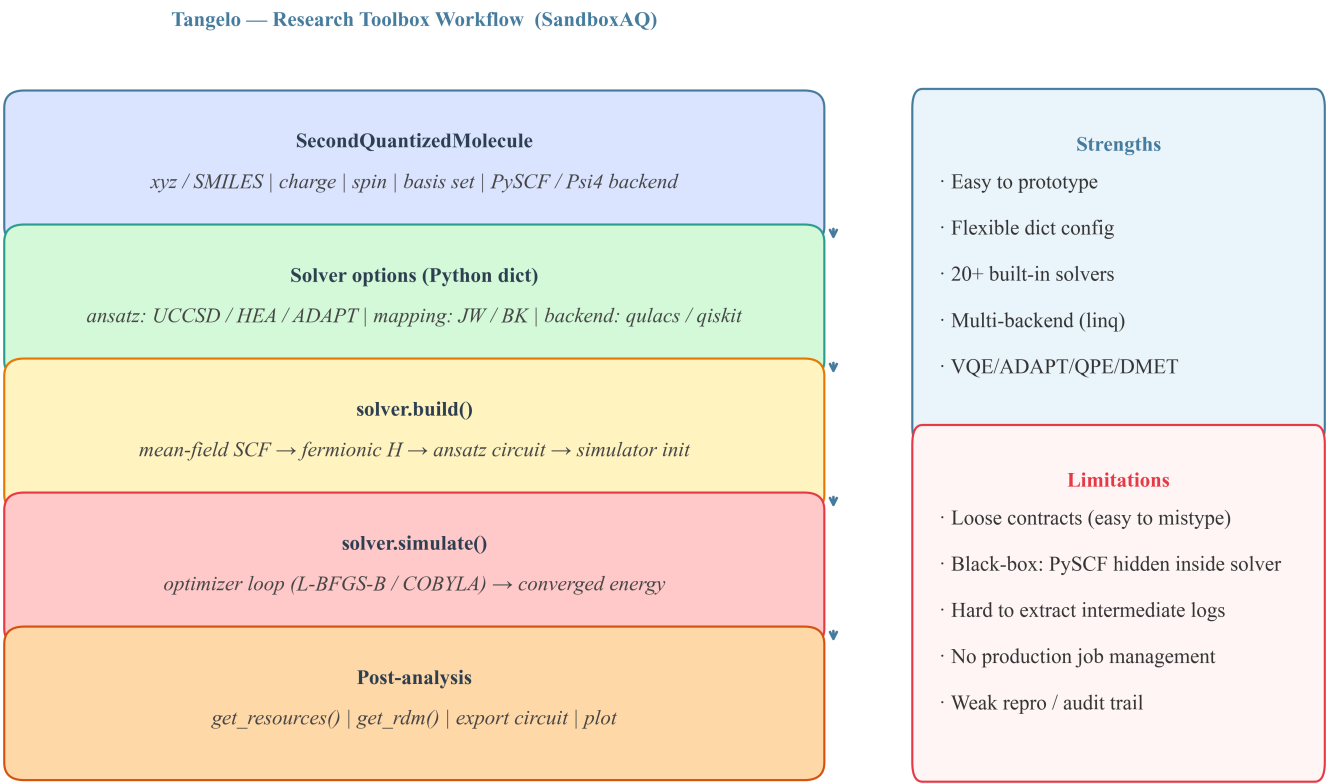
InQuanto 的核心是「真机与运营一体」：用强类型 API 串起 `Computable` / `Protocol`、TKET、Nexus 与 H-Series 后端，把哈密顿量推向可运营的真机测量：

```
1 from inquanto import FermionSpace, AlgorithmVQE, Protocol
2
3 fs = FermionSpace.from_pyscf(mol, active_space=(4, 4))
4 algo = AlgorithmVQE(fermion_space=fs, ansatz="UCCSD",
5                     optimizer="L-BFGS-B")
6 computable = algo.expectation_value(fs.hamiltonian)
7 result = Protocol(computable, backend="H1-1",
8                   shots=10000).run()
```

3.2 Tangelo: 开源研究工具包 (SandboxAQ)

[Tangelo](#) 是 SandboxAQ 维护的 Apache-2.0 开源工具包，定位研究与教学。

3.2.1 workflow 详解



Tangelo 的设计理念是"研究员至上"——用 Python 字典传参，几行代码跑通 VQE：

```
1 from tangelo.algorithms import VQESolver
2 from tangelo.molecule_library import mol_H2_sto3g
3
4 vqe = VQESolver({
5     "molecule": mol_H2_sto3g,
6     "ansatz": "UCCSD",
7     "qubit_mapping": "JW",
8     "backend_options": {"target": "qulacs"},
9 })
10 vqe.build()
11 energy = vqe.simulate()
```

3.3 竞品启发与工程取舍

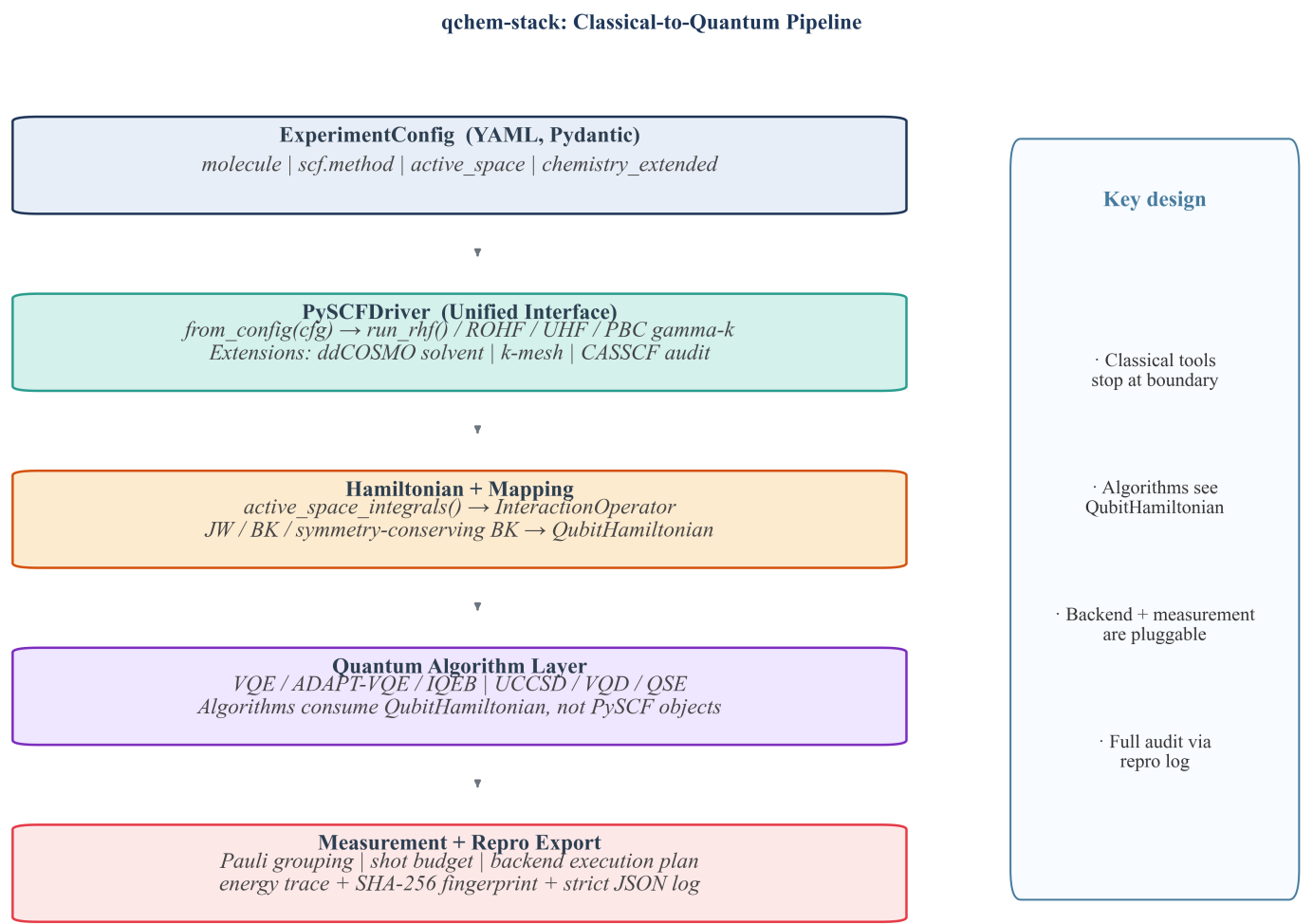
对比维度	InQuanto（优势/劣势）	Tangelo（优势/劣势）	qchem-stack 启发落地（取其长、避其短）
平台形态	优：闭环成熟； 劣：闭源锁定	优：开源灵活； 劣：交付链路轻	开源 + 插拔式架构，补齐工程交付
配置体验	优：规范；劣：机制不透明	优：字典上手快；劣：弱校验	YAML + Pydantic，失败早且明确
可审计性	优：企业流程完整；劣：核心黑盒	优：代码可见；劣：中间产物弱	显式 Driver 输出 + 哈密顿量指纹 + 分阶段导出
后端执行	优：硬件优化深；劣：绑定强	优：多后端方便；劣：偏研究态	BackendSpec 统一后端契约
作业管理	优：队列/配额成熟；劣：成本高	优：本地轻便；劣：缺少恢复	FastAPI + SQLite 打底异步作业
目标定位	企业级，但开放性不足	科研高效，但交付不足	可交付的开源量子化学流水线

一句话总结三者的本质差异：

- **InQuanto**：买一套完整的商业产品，省心但被锁死
- **Tangelo**：在 Notebook 里快速验证想法，灵活但难以交付
- **qchem-stack**：像搭工业流水线一样，严谨、可插拔、可审计

四、qchem-stack 现阶段工程：坚实骨架

我们的策略是：先搭严谨骨架，再逐步扩展算法能力。



整体流水线：YAML 契约 → 经典 Driver → **QubitHamiltonian** → 量子算法 → Pauli 测量协议 → Repro 导出。

4.1 配置即契约 (Config as Contract)

文件: `src/qchem_stack/config.py`

我们用 **Pydantic** + **YAML** 替代松散字典, 把非法参数拦截在 pipeline 启动阶段:

```
1 # 不合法的配置 → 立刻报错, 而不是在运行 3 小时后崩溃
2 ExperimentConfig.model_validate({
3     "molecule": {"symbols": ["H", "H"]},
4     "active_space": {
5         "n_active_orbitals": 100, # 超过实际轨道数
6         "fermion_qubit_mapping": "joddan_wigner" #
拼写错误
7     }
8 })
9 # → ValidationError: n_active_orbitals must be <=
total orbitals
1 #           fermion_qubit_mapping: must be
0 one of [...]
```

这让错误尽早暴露, 避免长时间运行后才发现参数拼错或越界。

4.2 经典计算化学: 统一契约 + 驱动层 (当前主路径: PySCF 全绑定)

经典计算不埋进 VQE Solver, 而是收敛到独立 Driver 层。当前默认 PySCF; 未来替换驱动或走 plugin 分支时, 量子算法仍消费同一份标准结果。

文件: `src/qchem_stack/chem/drivers/pyscf_driver.py`

```
1 @dataclass
2 class PySCFRHFResult:
3     mf: Any
4     e_tot: float
```



```

5     mo_energy: np.ndarray
6     molecular_system: MolecularSystem
7     driver_meta: dict
8
9     class PySCFDriver:
10         @classmethod
11         def from_config(cls, cfg: ExperimentConfig) ->
12         "PySCFDriver": ...
13
14         def run_rhf(self) -> PySCFRHFResult: ...
15         def run_rohf(self) -> PySCFRHFResult: ...
16         def run_uhf(self) -> PySCFRHFResult: ...
17         def run_pbc_rhf(self) -> PySCFRHFResult: ...

```

设计要点:

- 品牌隔离: PySCF 只停留在驱动层。
- 结果显式: SCF 能量、轨道、体系与元数据统一返回。
- 配置收口: RHF / ROHF / UHF / PBC 等由 YAML 进入 Driver。
- 可追溯: 关键输入输出写入 repro JSON。

4.3 哈密顿量构建与费米子映射

经典层产出统一结构后, 哈密顿量模块把 SCF 参考转为 **QubitHamiltonian**, 并从此与具体经典程序解耦。

文件: `src/qchem_stack/chem/hamiltonian.py`,
`src/qchem_stack/chem/fermion_mapping_registry.py`

```

1     def molecular_hamiltonian_from_pyscf(
2         rhf: PySCFRHFResult,
3         n_active_orbitals: int,

```

```

4     n_active_electrons: int,
5     fermion_qubit_mapping: Literal[
6         "jordan_wigner",
7         "bravyi_kitaev",
8         "symmetry_conserving_bravyi_kitaev"
9     ]
1    ) -> QubitHamiltonian:
0        """
1        活性空间积分提取 → 自旋轨道转换 → OpenFermion
2        InteractionOperator
1        → 量子比特哈密顿量。返回带 SHA-256 指纹的
3        QubitHamiltonian 对象。
1        """

```

可复现性保障：每个哈密顿量生成 SHA-256 指纹，用于校验相同配置下的算符一致性。

4.4 量子算法注册表

文件： `src/qchem_stack/quantum/algorithm_registry.py`,
`src/qchem_stack/quantum/algorithms/`

已接入统一注册表的算法：

算法	类型	说明
vqe	基态变分	UCCSD、HEA ansatz，多优化器支持
adapt	自适应变分	梯度驱动的 ansatz 自动生长
iqeb	基于 Pauli 选择的优化	比 ADAPT 更精细的泡利算符贡献排序
uccsd	化学启发 ansatz	经典量子化学的量子类比金标准
uccsd_trotter	Trotter 化分解	可配置步数的哈密顿量时间演化
vqd	激发态	Variational Quantum Deflation
qse	量子子空间展开	基态+激发态联算
sceom	方程运动	强关联激发态方法

切换算法只需修改 YAML 一行：

```
1 quantum:
2   algorithm: "adapt"    # 或 "iqeb", "uccsd", "vqd" ...
```

4.5 Pauli 测量协议（五阶段）

文件：src/qchem_stack/protocols/protocol.py

PauliAveragingProtocol 将期望值计算拆成五个阶段：

```
1 instantiate → build → compile → run → evaluate
2   (初始化)      (构建)      (编译)      (运行)      (后处理)
```

- **build**：按对易性分组，减少电路数
- **compile**：生成 CircuitIR，对接 TKET / Qiskit

- **run**: 插拔式后端执行
 - **evaluate**: 由 bit-string histogram 重建期望值
-

五、已跑通的示例与可视化结果

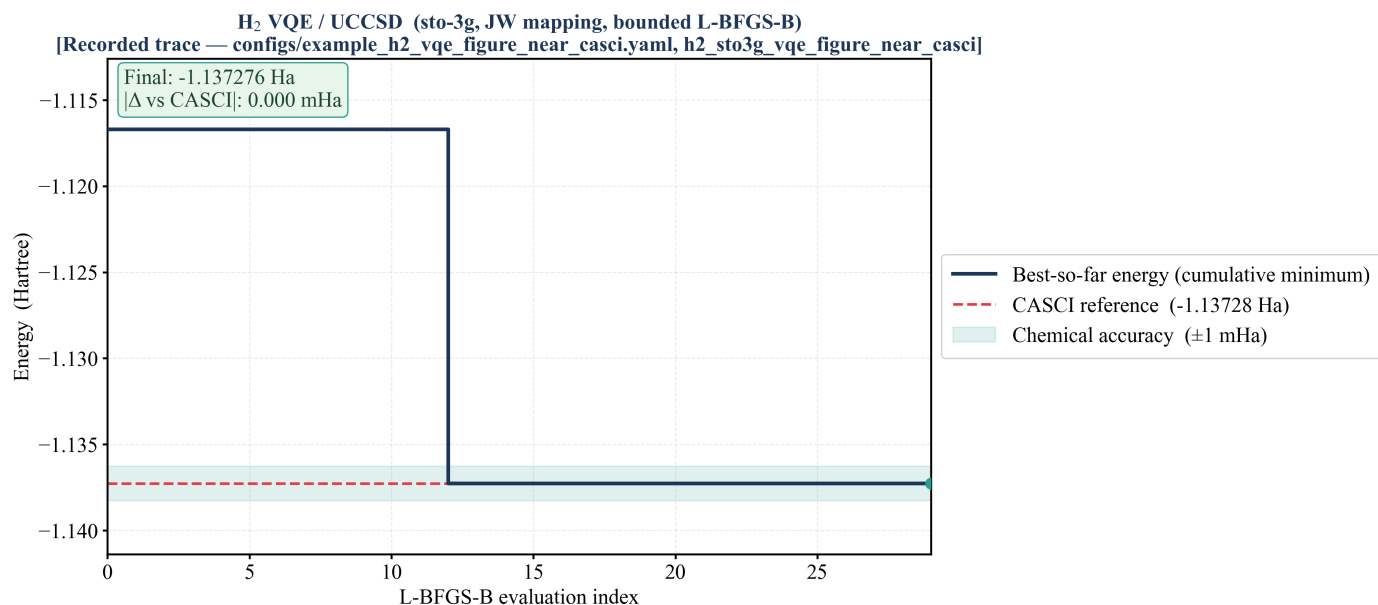
5.1 快速运行命令

```
1 # 基础 H2 VQE (statevector 精确模拟)
2 python scripts/smoke_pipeline.py
3
4 # Qiskit Aer 真实 shots 路径
5 python scripts/smoke_pipeline.py --qiskit-shots
6
7 # IQEB 算法变体
8 python scripts/smoke_pipeline.py --iqeb
9
1 # 激发态 VQD
10 python scripts/smoke_pipeline.py --excited-only
```

5.2 H₂ VQE 优化收敛

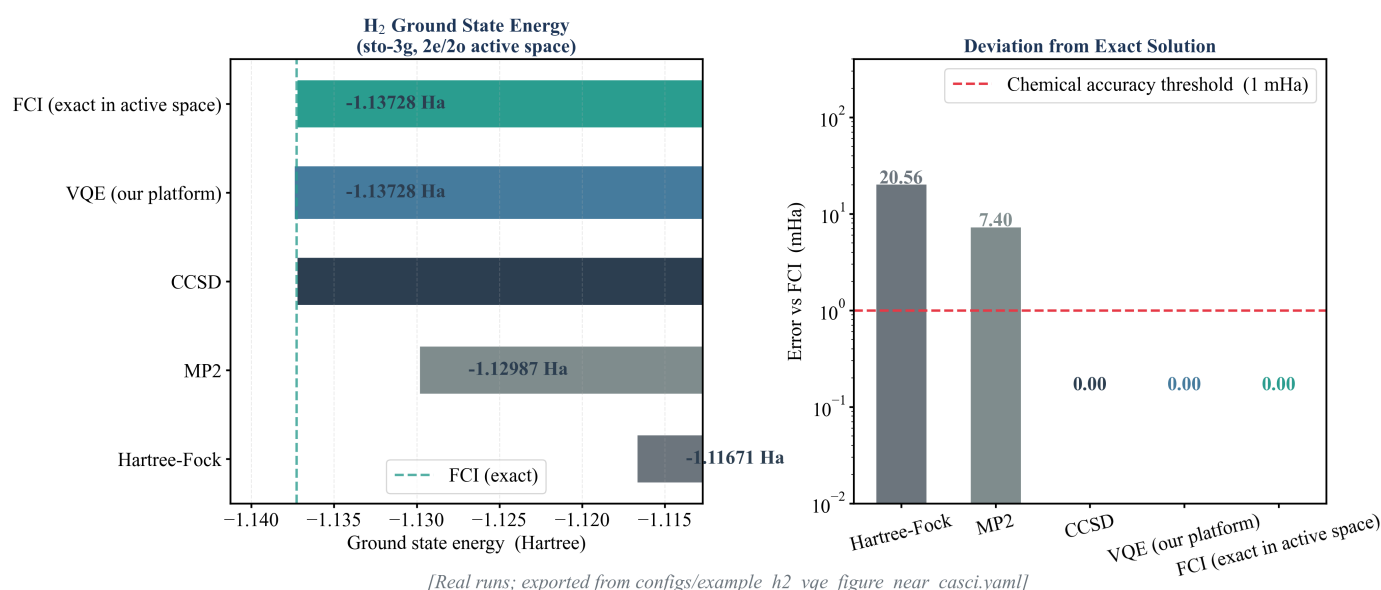
配置文件（出图专用）：

`configs/example_h2_vqe_figure_near_casci.yaml` (H₂ /sto-3g, UCCSD, 关闭 Pauli protocol)



5.3 经典方法 vs 量子方法能量对比（实跑导出）

同一体系（H₂ /sto-3g）的能量与误差对比：



方法	能量 (Ha)	误差 vs FCI (mHa)	来源
Hartree-Fock	-1.116714	20.562	export_classical_quantum_comparison_data.py (PySCF RHF)
MP2	-1.129872	7.404	同上 (PySCF MP2)
CCSD	-1.137276	0.0002	同上 (PySCF CCSD)
VQE (我们的平台)	-1.137276	≈0	同上 (UCCSD + L-BFGS-B 实跑最优值)
FCI (active-space CASCI)	-1.137276	0	同上 (PySCF CASCI, 2e/2o)

5.4 费米子-量子比特映射（提要）

平台通过 `active_space.fermion_qubit_mapping` 支持 JW、BK 与对称守恒 BK / SCBK。

附录 A. 最小运行示例

```
1 # 安装（开发模式）
2 pip install -e ".[dev]"
3
4 # 运行基础流水线（需 PySCF）
5 pip install pyscf openfermion
6 python scripts/smoke_pipeline.py
7
8 # 启动本地 API 服务
9 uvicorn qchem_stack.api.app:app --host 127.0.0.1 --
  port 8000
1
10 # 预览工作流（无需真实计算）
11 curl http://127.0.0.1:8000/workflow-preview
```

附录 B. 标准配置文件示例（`configs/example_h2.yaml`）

```
1 schema_version: "1"
2 experiment_id: "h2_sto3g_vqe_demo"
3
4 molecule:
5   symbols: ["H", "H"]
6   coordinates_bohr: [[0.0, 0.0, 0.0], [0.0, 0.0,
  1.4]]
7   charge: 0
8   multiplicity: 1
```

```
9     basis: "sto-3g"
1
0 scf:
1     driver: "pyscf"
2     method: "RHF"
3
4 active_space:
5     n_active_orbitals: 2
6     n_active_electrons: 2
7     fermion_qubit_mapping: "jordan_wigner"
8
9 quantum:
0     algorithm: "vqe"
1     vqe_depth: 1
2     use_pauli_protocol: true
3
4 backend:
5     provider: "statevector"
6     shots_per_circuit: 1024
```